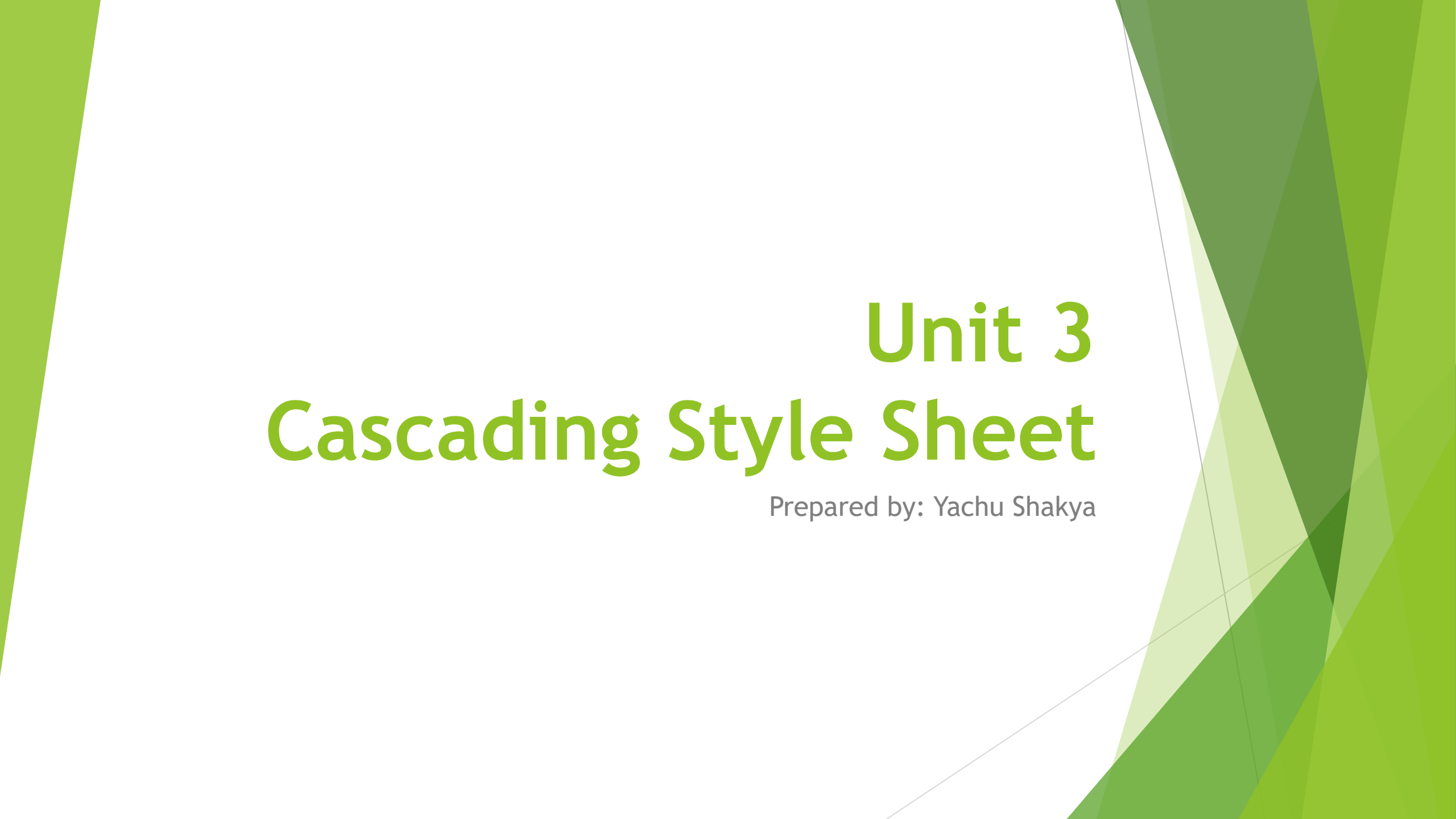


The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Unit 3

Cascading Style Sheet

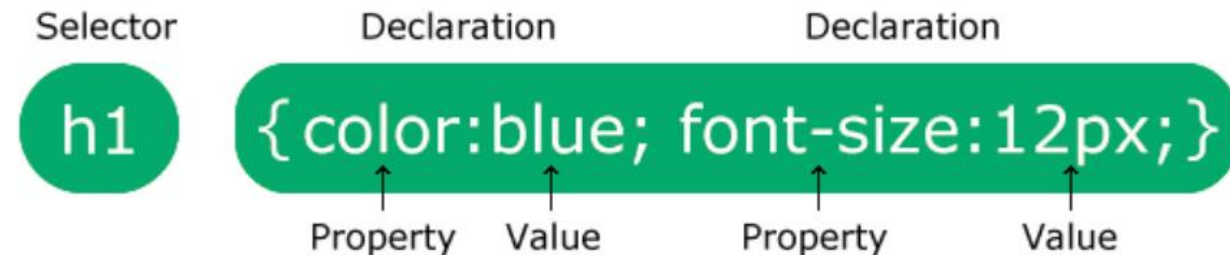
Prepared by: Yachu Shakya

CSS (Cascading Style Sheets)

- ▶ CSS (Cascading Style Sheets) is a stylesheet language used for describing the presentation and formatting of a document written in HTML or XML. It is used to control the layout, design, and appearance of web pages.
- ▶ CSS works by selecting HTML elements and applying specific styles to them. These styles define how the elements should be displayed on the screen, including properties such as color, font, size, spacing, and positioning.

CSS Syntax

- ▶ The selector points to the HTML element you want to style.
- ▶ The declaration block contains one or more declarations separated by semicolons.
- ▶ Each declaration includes a CSS property name and a value, separated by a colon.
- ▶ Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.



How To Add CSS

There are three ways of inserting a style sheet:

- ▶ **Inline CSS**
- ▶ **Internal CSS**
- ▶ **External CSS**

Inline CSS

- ▶ Inline CSS is a method of applying styles directly to individual HTML elements
`<p style="color: blue; font-size: 18px; text-align: center;">This is a paragraph with inline CSS.</p>`

In the example above, the `<p>` element has inline CSS applied to it. The style attribute contains multiple CSS property-value pairs separated by semicolons.

Inline CSS

- ▶ The color property is set to "blue," which will make the text inside the paragraph appear in blue.
- ▶ The font-size property is set to "18px," which will make the text size larger than the default size.
- ▶ The text-align property is set to "center," which will align the text in the center of the paragraph.
- ▶ You can add or modify CSS properties within the style attribute to customize the appearance of the element. However, keep in mind that using inline CSS for extensive styling can become cumbersome and harder to maintain compared to using external or internal CSS stylesheets.

Internal CSS

Internal CSS is a method of defining styles within the `<style>` tags in the `<head>` section of an HTML document. Here's an example of internal CSS:

```
<!DOCTYPE html>
<html>
<head>
  <style>
    p {
      color: blue;
      font-size: 18px;
      text-align: center;
    }
  </style>
```

```
</head>
<body>
  <p>This is a paragraph with internal CSS.</p>
</body>
</html>
```

Internal CSS

In the example above, the styles for the `<p>` element are defined within the `<style>` tags in the `<head>` section of the HTML document.

- ▶ The `p` selector targets all `<p>` elements.
- ▶ The `color` property is set to "blue," which will make the text inside the paragraphs appear in blue.
- ▶ The `font-size` property is set to "18px," which will make the text size larger than the default size.
- ▶ The `text-align` property is set to "center," which will align the text in the center of the paragraphs.
- ▶ By placing the styles within the `<style>` tags in the `<head>` section, the defined styles will be applied to all `<p>` elements within the HTML document

External CSS

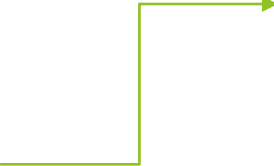
External CSS is a method of defining styles in a separate CSS file and linking it to an HTML document using the `<link>` tag. Here's an example of external CSS: Suppose you have an HTML file named "index.html" and a CSS file named "styles.css" located in the same directory.

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <p>This is a paragraph with external
  CSS.</p>
</body>
</html>
```

styles.css

```
p {
  color: blue;
  font-size: 18px;
  text-align: center;
}
```

A green line starts from the right side of the `<link>` tag in the HTML code, moves horizontally to the right, then turns 90 degrees upwards, and finally turns 90 degrees to the right, ending with an arrow pointing to the `styles.css` header.

External CSS

In the example above, the CSS styles are defined in a separate file named "styles.css". The `<link>` tag in the `<head>` section of the HTML document links the CSS file to the HTML file using the href attribute.

The styles defined in "styles.css" are applied to all `<p>` elements in the HTML document. The styles include:

- ▶ The color property is set to "blue," which will make the text inside the paragraphs appear in blue.
- ▶ The font-size property is set to "18px," which will make the text size larger than the default size.
- ▶ The text-align property is set to "center," which will align the text in the center of the paragraphs.

Question

What is CSS(Cascading Style Sheet)? Define CSS Syntax. What are the three ways to insert a stylesheet. Explain with example

Writing comments in CSS

- ▶ To add a comment in a CSS file, you can use the following syntax:
- ▶ `/* This is a comment in CSS */`
- ▶ Any text between `/*` and `*/` will be considered a comment and will be ignored by the browser when rendering the page. Here's an example of how you can use comments in a CSS file:

```
/* This is a comment */
body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2; /* Background color for the body */
}

/*
  This is a multiline comment
  You can add multiple lines of comments within the comment block
*/
h1 {
  color: #333; /* Text color for headings */
}
```

Writing comments in HTML

- ▶ Anything between `<!--` and `-->` will be treated as a comment and will be ignored by the browser.
- ▶ Both HTML and CSS comments play an important role in improving code readability, facilitating collaboration, and making future modifications or debugging easier. It's good practice to use comments effectively in both HTML and CSS to provide clarity and context to your code.

```
<html>
<head>
  <title>My Website</title>
</head>
<body>
  <!-- This is a comment -->
  <h1>Welcome to My Website</h1>
  <p>This is the homepage.</p>

  <!--
    This is a multiline comment.
    You can add multiple lines of comments within the comment block.
  -->

  <footer>
    <p>&copy; 2023 My Website. All rights reserved.</p>
  </footer>
```

Types of selector

In CSS, there are several types of selectors that allow you to target and style specific elements in an HTML document. Here are some of the most commonly used selector types:

1. Simple selector
2. Universal selector
3. Grouping selector
4. Attribute selector
5. Pseudo class selector
6. Pseudo element selector
7. Combinators selector

1. Simple selector

1.1 Element Selector: Selects elements based on their HTML tag name. For example:

```
p {  
  color: blue;  
}
```

- This rule selects all <p> elements and applies the specified style.

1.2 Class Selector: Selects elements based on the value of their class attribute. For example:

```
.highlight {  
  background-color: yellow;  
}
```

- This rule selects all elements with class="highlight" and applies the specified style.

1.3 ID Selector: Selects a single element based on its unique id attribute. For example:

```
#header {  
  font-size: 24px;  
}
```

This rule selects the element with id="header" and applies the specified style.

2. Universal selector

- ▶ The universal selector *in CSS is a special type of selector that matches any element in the HTML document. It is represented by an asterisk* . When used in a CSS rule, the universal selector applies that rule to every element on the page.
- ▶ Here's an example of how the universal selector can be used in CSS:

```
* {  
  margin: 0;  
  padding: 0;  
}
```

In this example, the universal selector is used to target all elements on the page and set their margin and padding properties to zero. This rule essentially resets the margin and padding of all elements, providing a consistent starting point for styling.

3. Grouping selector

- ▶ In CSS, a grouping selector allows you to apply styles to multiple selectors at once. It enables you to group together multiple selectors and define the same styles for all of them. Grouping selectors can be useful for reducing code repetition and applying consistent styles to related elements. Here's how you can use grouping selectors:
- ▶ For instance, if you want to apply the same font color and font size to all `<h1>`, `<h2>`, and `<h3>` elements, you can use a grouping selector like this:

```
h1, h2, h3 {  
  color: #333;  
  font-size: 24px;  
}
```

3. Grouping selector

Grouping selectors can also be combined with other types of selectors to create more specific rules. For example, you can use a combination of class selectors and grouping selectors like this:

```
.header, .footer, .sidebar {  
  background-color: #f2f2f2;  
  padding: 10px;  
}
```

In this case, the styles defined within the curly braces will be applied to elements with the class `.header`, `.footer`, or `.sidebar`.

Using grouping selectors can help streamline your CSS code by applying shared styles to multiple selectors at once, making it easier to maintain and update your styles.

4. Attribute Selector

- ▶ In CSS, an attribute selector allows you to target elements based on the presence or value of an attribute. It provides a way to select elements that match specific attribute conditions. Attribute selectors are denoted by square brackets ([]). Here are some examples of how attribute selectors can be used:

4.1 CSS [attribute] Selector

- ▶ The [attribute] selector is used to select elements with a specified attribute.
- ▶ The following example selects all <a> elements with a target attribute:

```
a[target] {  
  background-color: yellow;  
}
```

4.2 CSS [attribute="value"] Selector

- ▶ The [attribute="value"] selector is used to select elements with a specified attribute and value.
- ▶ The following example selects all <input> elements with a type="text" attribute:

```
input[type="text"] {  
  border: 1px solid gray;  
}
```

- ▶ This rule selects all <input> elements with type="text" and applies the specified style.

4.3 CSS [attribute~="value"] Selector

- ▶ The [attribute~="value"] selector is used to select elements with an attribute value containing a specified word.
- ▶ The following example selects all elements with a title attribute that contains a space-separated list of words, one of which is "flower":

```
[title~="flower"] {  
  border: 5px solid yellow;  
}
```

- ▶ The example above will match elements with title="flower", title="summer flower", and title="flower new", but not title="my-flower".

4.4 CSS [attribute | ="value"] Selector

- ▶ The [attribute | ="value"] selector is used to select elements with the specified attribute, whose value can be exactly the specified value, or the specified value followed by a hyphen (-).
- ▶ Note: The value has to be a whole word, either alone, like class="top", or followed by a hyphen(-), like class="top-text".
- ▶ Example:

```
[class | ="top"] {  
  background: yellow;  
}
```

4.5 CSS [attribute^="value"] Selector

- ▶ The [attribute^="value"] selector is used to select elements with the specified attribute, whose value starts with the specified value.
- ▶ The following example selects all elements with a class attribute value that starts with "top":
- ▶ **Note:** The value does not have to be a whole word!

```
[class^="top"] {  
  background: yellow;  
}
```

4.6 CSS [attribute\$="value"] Selector

- ▶ The [attribute\$="value"] selector is used to select elements whose attribute value ends with a specified value.
- ▶ The following example selects all elements with a class attribute value that ends with "test":
- ▶ Note: The value does not have to be a whole word!

```
[class$="test"] {  
  background: yellow;  
}
```

4.7 CSS [attribute*="value"] Selector

- ▶ The [attribute*="value"] selector is used to select elements whose attribute value contains a specified value.
- ▶ The following example selects all elements with a class attribute value that contains "te":
- ▶ Note: The value does not have to be a whole word!

```
[class*="te"] {  
  background: yellow;  
}
```

5. Pseudo-class selector

A pseudo-class is used to define a special state of an element.

For example, it can be used to:

- ▶ Style an element when a user mouse over it
- ▶ Style visited and unvisited links differently
- ▶ Style an element when it gets focus
- ▶ Syntax:

```
selector:pseudo-class {  
  property: value;  
}
```

5. Pseudo-class selector

CSS pseudo-class	
:link	It adds style to unvisited link.
:visited	It adds style to a visited link. TutorialBrain.com
:hover	It adds style to element when we mouse over it.
:active	It adds style to the active link.
:focus	It adds style to element when it has focus.
:first-child	This class adds style to the first child of the element.
:last-child	This class adds style to the second child of the element.
:lang	It defines the language of the specified element.

6. Pseudo-element selector

- ▶ A CSS pseudo-element is used to style specified parts of an element.
- ▶ For example, it can be used to:
- ▶ Style the first letter, or line, of an element
- ▶ Insert content before, or after, the content of an element
- ▶ Syntax:

```
selector::pseudo-element {  
  property: value;  
}
```

6. Pseudo-element selector

Selector	Example	Example description
<u>::after</u>	p::after	Insert something after the content of each <p> element
<u>::before</u>	p::before	Insert something before the content of each <p> element
<u>::first-letter</u>	p::first-letter	Selects the first letter of each <p> element
<u>::first-line</u>	p::first-line	Selects the first line of each <p> element
<u>::marker</u>	::marker	Selects the markers of list items
<u>::selection</u>	p::selection	Selects the portion of an element that is selected by a user

7. Combinators selector

A CSS selector can contain more than one simple selector. Between the simple selectors, we can include a combinator.

There are four different combinators in CSS:

- ▶ descendant selector (space)
- ▶ child selector (>)
- ▶ adjacent sibling selector (+)
- ▶ general sibling selector (~)

7.1 Descendant Selector

- ▶ The descendant selector matches all elements that are descendants of a specified element.
- ▶ The following example selects all <p> elements inside <div> elements:

```
div p {  
  background-color: yellow;  
}
```

7.2 Child Selector (>)

- ▶ The child selector selects all elements that are the children of a specified element.
- ▶ The following example selects all <p> elements that are children of a <div> element:

```
div > p {  
  background-color: yellow;  
}
```

7.3 Adjacent Sibling Selector (+)

- ▶ The adjacent sibling selector is used to select an element that is directly after another specific element.
- ▶ Sibling elements must have the same parent element, and "adjacent" means "immediately following".
- ▶ The following example selects the first `<p>` element that are placed immediately after `<div>` elements:

```
div + p {  
  background-color: yellow;  
}
```

7.4 General Sibling Selector (~)

- ▶ The general sibling selector selects all elements that are next siblings of a specified element.
- ▶ The following example selects all <p> elements that are next siblings of <div> elements:

```
div ~ p {  
  background-color: yellow;  
}
```

Background-color

In CSS (Cascading Style Sheets), you can set the background color of an element using the background-color property. This property allows you to specify a color value to be used as the background of the element. Here's how you can use it:

```
.selector {  
    background-color: value;  
}
```

In the above code snippet, replace `.selector` with the appropriate selector for your element. It can be a class name (prefixed with a dot), an ID (prefixed with a hash), or an HTML element name.

Background-color

Replace value with the desired color for the background. There are several ways to define a color value in CSS:

- ▶ **Named Colors:** You can use named colors such as red, blue, green, etc.
- ▶ **Hexadecimal Colors:** Hex values are specified using a pound sign (#) followed by six hexadecimal digits. For example, #FF0000 represents red, #00FF00 represents green, and #0000FF represents blue.
- ▶ **RGB Colors:** RGB (Red, Green, Blue) values allow you to define a color using the intensity of each primary color. It is represented as `rgb(red, green, blue)`, where each component value ranges from 0 to 255. For example, `rgb(255, 0, 0)` represents red, `rgb(0, 255, 0)` represents green, and `rgb(0, 0, 255)` represents blue.
- ▶ **HSL:** HSL stands for Hue, Saturation and Lightness.
 - ▶ An HSL color value is specified with: `hsl(hue, saturation, lightness)`.
 - 1. Hue is a degree on the color wheel (from 0 to 360):
 - 1. 0 (or 360) is red
 - 2. 120 is green
 - 3. 240 is blue
 - 2. Saturation is a percentage value: 100% is the full color.
 - 3. Lightness is also a percentage; 0% is dark (black) and 100% is white.

Example

- ▶ `.example { background-color: red; }`
- ▶ `.example { background-color: #00FF00; }`
- ▶ `.example { background-color: rgb(0, 0, 255); }`

Background Image

- ▶ In CSS, you can set a background image for an element using the background-image property. This property allows you to specify the URL of the image you want to use as the background. Here's how you can use it:

```
.selector {  
  background-image: url(image-url);  
}
```

- ▶ Replace .selector with the appropriate selector for your element, as described in the previous response.
- ▶ Replace image-url with the URL or relative path to the image you want to use as the background. For example, if the image is in the same directory as your CSS file, you can simply provide the filename: url(image.jpg). If the image is hosted on a different server, you'll need to provide the full URL, such as url(http://example.com/image.jpg)

Background Image

Additionally, you can use the following CSS properties to control the behavior of the background image:

- ▶ **background-repeat:** Specifies whether and how the background image should repeat. Possible values include repeat (default), repeat-x, repeat-y, and no-repeat.
- ▶ **background-position:** Specifies the starting position of the background image. You can use keywords like top, bottom, left, right, center, or specific measurements such as 10px 20px.
- ▶ **background-size:** Specifies the size of the background image. You can use values like cover (scales the image to cover the entire background), contain (scales the image to fit within the background), or specific measurements such as 200px 100px.

Background Image

```
.example {  
  background-image: url(image.jpg);  
  background-repeat: no-repeat;  
  background-position: center;  
  background-size: cover;  
}
```

In the above example, the image `image.jpg` will be used as the background for the elements with the class `.example`. It won't repeat, and it will be positioned at the center of the element. The image will be scaled to cover the entire background area.

CSS Border

In CSS (Cascading Style Sheets), you can use the border property to add borders to elements on a webpage. The border property allows you to control the width, style, and color of the border.

Here's the basic syntax for the border property:

```
selector {  
  border: width style color;  
}
```

CSS Border

- ▶ **width** specifies the width of the border. It can be set using a length value like pixels (e.g., 1px, 2px), or you can use keywords such as thin, medium, or thick. You can also set different widths for different sides of the border using the border-width property.
- ▶ **style** defines the style of the border. It can be set using keywords such as none, solid, dashed, dotted, double, groove, ridge, inset, or outset. You can also use the border-style property to set different styles for different sides of the border.
- ▶ **color** determines the color of the border. It can be set using a color name (e.g., red, blue), a hexadecimal value (e.g., #FF0000, #00F), an RGB value (e.g., rgb(255, 0, 0)), or an HSL value (e.g., hsl(0, 100%, 50%)). You can also use the border-color property to set different colors for different sides of the border.
- ▶ Example:

```
div {  
  border: 2px solid red;  
}
```

CSS Border Style

```
p.dotted {border-style: dotted;}  
p.dashed {border-style: dashed;}  
p.solid {border-style: solid;}  
p.double {border-style: double;}  
p.groove {border-style: groove;}  
p.ridge {border-style: ridge;}  
p.inset {border-style: inset;}  
p.outset {border-style: outset;}  
p.none {border-style: none;}  
p.hidden {border-style: hidden;}  
p.mix {border-style: dotted dashed solid double;}
```

A dotted border.

A dashed border.

A solid border.

A double border.

A groove border. The effect depends on the border-color value.

A ridge border. The effect depends on the border-color value.

An inset border. The effect depends on the border-color value.

An outset border. The effect depends on the border-color value.

No border.

A hidden border.

A mixed border.

CSS border individual side

- ▶ From the examples on the previous pages, you have seen that it is possible to specify a different border for each side.
- ▶ In CSS, there are also properties for specifying each of the borders (top, right, bottom, and left):

```
p {  
  border-top-style: dotted;  
  border-right-style: solid;  
  border-bottom-style: dotted;  
  border-left-style: solid;  
}
```

Different Border Styles

CSS Border Width

- ▶ The border-width property specifies the width of the four borders.
- ▶ The width can be set as a specific size (in px, pt, cm, em, etc) or by using one of the three pre-defined values: thin, medium, or thick:

```
p.one {  
  border-style: solid;  
  border-width: 5px;  
}
```

5px border-width

```
p.two {  
  border-style: solid;  
  border-width: medium;  
}
```

medium border-width

```
p.three {  
  border-style: dotted;  
  border-width: 2px;  
}
```

2px border-width

```
p.four {  
  border-style: dotted;  
  border-width: thick;  
}
```

thick border-width

CSS Border Color

- ▶ The border-color property is used to set the color of the four borders.
- ▶ The color can be set by:
 - ▶ name - specify a color name, like "red"
 - ▶ HEX - specify a HEX value, like "#ff0000"
 - ▶ RGB - specify a RGB value, like "rgb(255,0,0)"
 - ▶ HSL - specify a HSL value, like "hsl(0, 100%, 50%)"

CSS Rounded Borders

- The border-radius property is used to add rounded borders to an element:

```
p.normal {  
  border: 2px solid red;  
  padding: 5px;  
}
```

```
p.round1 {  
  border: 2px solid red;  
  border-radius: 5px;  
  padding: 5px;  
}
```

```
p.round2 {  
  border: 2px solid red;  
  border-radius: 8px;  
  padding: 5px;  
}
```

```
p.round3 {  
  border: 2px solid red;  
  border-radius: 12px;  
  padding: 5px;  
}
```

Normal border

Round border

Rounder border

Roudest border

CSS Height/Width

- ▶ In CSS, you can set the height and width of an element using the height and width properties. These properties allow you to specify the dimensions of an element in various units of measurement.
- ▶ Here's an example of how you can set the height and width of an element in CSS:
- ▶ Example:

```
.element { height: 200px; width: 300px; }
```

CSS Text

In CSS, you can style text in various ways using different properties. Here are some common CSS properties used for styling text:

- ▶ The **color** property is used to set the color of a text.
- ▶ The **direction** property is used to set the text direction.
- ▶ The **letter-spacing** property is used to add or subtract space between the letters that make up a word.
- ▶ The **word-spacing** property is used to add or subtract space between the words of a sentence.

CSS Text

- ▶ The **text-align** property is used to align the text of a document.
- ▶ The **text-decoration** property is used to underline, overline, and strikethrough text.
- ▶ The **text-transform** property is used to capitalize text or convert text to uppercase or lowercase letters.
- ▶ The **text-shadow** property is used to set the text shadow around a text.

Example

```
<html>
<head>
<style>
h1 {
  color: #ff0000;
  letter-spacing: 2px;
  word-spacing: 4px;
  text-align: center;
  text-decoration: underline;
  text-transform: uppercase;
  text-shadow: 2px 2px 4px blue;
}
</style>
<body>
<h1> This is Example of CSS Text Styling. </h1>
</body>
</html>
```

THIS IS EXAMPLE OF CSS TEXT STYLING.

CSS Font

- ▶ In CSS, you can control the font styles and properties of your web page using the font property. The font property allows you to set various aspects of the font, such as the font family, size, weight, style, and more.
- ▶ Here's the general syntax for the font property:

```
selector {
```

```
    font: font-style font-variant font-weight font-size/line-height font-family;
```

```
}
```

CSS Font property

Let's break down each part of the font property:

- ▶ **font-style:** Specifies the style of the font, such as "normal" (default), "italic," or "oblique."
- ▶ **font-variant:** Specifies whether the font should be displayed in small caps. Common values are "normal" (default) or "small-caps."
- ▶ **font-weight:** Specifies the weight or thickness of the font, such as "normal" (default), "bold," "bolder," "lighter," or numeric values like 100, 200, 300, etc.
- ▶ **font-size/line-height:** Specifies the size of the font and the height of each line. You can set the size in pixels (px), ems (em), rems (rem), or other length units. Line height can be set as a number, percentage, or length unit.
- ▶ **font-family:** Specifies the font family or multiple font families to be used. You can list multiple font families separated by commas, and the browser will use the first available font on the user's system.

Example

```
<html>
<head>
<style>
h1 {
  font-family: Arial, sans-serif;
  font-size: 16px;
  font-weight: bold;
  font-style: italic;
  line-height: 3.5;
  color: #FF0000;
  text-decoration: underline;
  text-transform: uppercase;
}

</style>
<body>
<h1> This is Example of CSS Text Styling. </h1>
<h1> This is Example of CSS Text Styling. </h1>
</body>
</html>
```

THIS IS EXAMPLE OF CSS TEXT STYLING.

THIS IS EXAMPLE OF CSS TEXT STYLING.

CSS List Styling

- In CSS, you can style lists using the `list-style` property to control the appearance of bullets or numbering for unordered and ordered lists, respectively. Additionally, you can use other properties to customize the layout and presentation of list items. Here are some common CSS properties for list styling:

Property	Description
<u><code>list-style</code></u>	Sets all the properties for a list in one declaration
<u><code>list-style-image</code></u>	Specifies an image as the list-item marker
<u><code>list-style-position</code></u>	Specifies the position of the list-item markers (bullet points)
<u><code>list-style-type</code></u>	Specifies the type of list-item marker

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
ul {
  list-style-type: square;
  list-style-position: inside;
  list-style-image: url('icon.png');
}
</style>
</head>
<body>

<h2>The list-style-image Property</h2>

<p>The list-style-image property specifies an image as the list item marker:</p>

<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Coca Cola</li>
</ul>

</body>
</html>
```

The list-style-image Property

The list-style-image property specifies an image as the list item marker:

- ☼ Coffee
- ☼ Tea
- ☼ Coca Cola

Example

"list-style-position: outside;" means that the bullet points will be outside the list item. The start of each line of a list item will be aligned vertically. This is default:

- | |
|---|
| • Coffee - A brewed drink prepared from roasted coffee beans... |
| • Tea |
| • Coca-cola |

"list-style-position: inside;" means that the bullet points will be inside the list item. As it is part of the list item, it will be part of the text and push the text at the start:

- | |
|---|
| • Coffee - A brewed drink prepared from roasted coffee beans... |
| • Tea |
| • Coca-cola |

CSS table

- ▶ **CSS tables** are used to style HTML tables, making them look neat and organized for clear data presentation.
- Add borders, spacing, and colors to enhance table design.
- Create responsive tables that adjust for all screen sizes.

Properties:

border: 1px solid black

margin: 25px

border-collapse: collapse

border-spacing

width:

height:

vertical-align:

CSS max-width/min-width property

- ▶ In CSS, the max-width property is used to set the maximum width that an element can have. It limits the width of an element to a specified value, preventing it from exceeding that value.
- ▶ It is commonly used to make sure that elements do not become too wide on larger screens or when the browser window is resized. Using max-width, will improve the browser's handling of small windows. This is important when making a site usable on small devices.

Example

```
<style>
.container1 {
  max-width: 600px;
  margin: auto; /* Center the container */
  background-color: lightblue;
  padding: 20px;
}
.container2 {
  width: 600px;
  margin: auto; /* Center the container */
  background-color: lightblue;
  padding: 20px;
}
</style>
```

Max Width Example

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin ac maximus nisi. Nullam id nisi vestibulum, vulputate leo ac, convallis ex. Duis eget congue arcu. Fusce congue luctus diam, at tristique enim. Phasellus vel ullamcorper dolor. Nam at malesuada nisl, vitae bibendum lectus. Suspendisse a ligula id urna aliquet aliquam. Integer vitae dui efficitur, gravida felis vel, rhoncus felis. Donec auctor venenatis fringilla. Sed pellentesque eleifend lorem id bibendum. Nam eget efficitur sem.

 Example Image

Without Max Width Example

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin ac maximus nisi. Nullam id nisi vestibulum, vulputate leo ac, convallis ex. Duis eget congue arcu. Fusce congue luctus diam, at tristique enim. Phasellus vel ullamcorper dolor. Nam at malesuada nisl, vitae bibendum lectus. Suspendisse a ligula id urna aliquet aliquam. Integer vitae dui efficitur, gravida felis vel, rhoncus felis. Donec auctor venenatis fringilla. Sed pellentesque eleifend lorem id bibendum. Nam eget efficitur sem.

 Example Image

The auto Value

- ▶ You can set the margin property to auto to horizontally center the element within its container.
- ▶ The element will then take up the specified width, and the remaining space will be split equally between the left and right margins.

```
div {  
  width: 300px;  
  margin: auto;  
  border: 1px solid red;  
}
```

The inherit Value

- This example lets the left margin of the `<p class="ex1">` element be inherited from the parent element (`<div>`):

```
div {  
  border: 1px solid red;  
  margin-left: 100px;  
}  
  
p.ex1 {  
  margin-left: inherit;  
}  
</style>  
</head>  
<body>  
  
<h2>Use of the inherit value</h2>  
<p>Let the left margin be inherited from the parent element:</p>  
  
<div>  
<p class="ex1">This paragraph has an inherited left margin (from the  
div element).</p>  
</div>
```

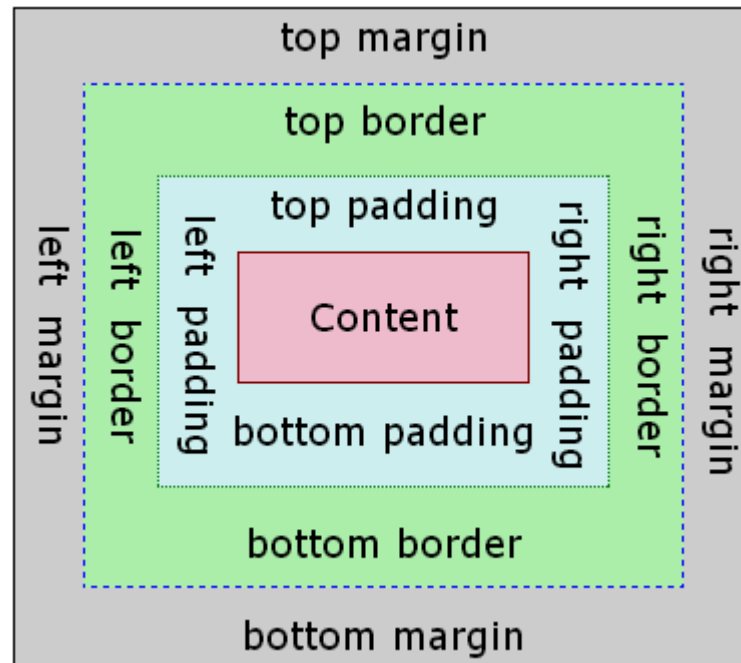
Use of the inherit value

Let the left margin be inherited from the parent element:

This paragraph has an inherited left margin (from the div element).

CSS Box Model

- The CSS box model is a fundamental concept that describes the layout and spacing of elements on a webpage. It consists of four components: content, padding, border, and margin. Understanding the box model is crucial for controlling the dimensions and spacing of elements.



CSS Margin

- ▶ In CSS (Cascading Style Sheets), the margin property is used to control the space outside an element's border. It defines the gap or distance between an element and its neighboring elements.
- ▶ The margin property allows you to set the margin on all four sides of an element at once or specify different values for each side.
- ▶ Here's the basic syntax for the margin property:

```
selector {  
  margin: value;  
}
```

CSS Margin

Value can be specified in various formats:

- ▶ **A single value:** This value will be applied to all four sides of the element's margin. For example, `margin: 10px;` sets a margin of 10 pixels on all sides.
- ▶ **Two values:** The first value applies to the top and bottom margins, while the second value applies to the left and right margins. For example, `margin: 10px 20px;` sets a margin of 10 pixels for the top and bottom, and 20 pixels for the left and right.
- ▶ **Three values:** The first value applies to the top margin, the second value applies to the left and right margins, and the third value applies to the bottom margin. For example, `margin: 10px 20px 15px;` sets a margin of 10 pixels for the top, 20 pixels for the left and right, and 15 pixels for the bottom.
- ▶ **Four values:** Each value applies to the top, right, bottom, and left margins, respectively, in clockwise order. For example, `margin: 10px 20px 15px 5px;` sets a margin of 10 pixels for the top, 20 pixels for the right, 15 pixels for the bottom, and 5 pixels for the left.

All CSS Margin Properties

Property	Description
<u>margin</u>	A shorthand property for setting all the margin properties in one declaration
<u>margin-bottom</u>	Sets the bottom margin of an element
<u>margin-left</u>	Sets the left margin of an element
<u>margin-right</u>	Sets the right margin of an element
<u>margin-top</u>	Sets the top margin of an element

CSS Padding

- ▶ CSS padding is a property that allows you to specify the space between the content of an element and its borders. It adds space inside an element, pushing the content away from the edges. The padding is applied to all four sides of the element, but you can also specify individual values for each side if desired.
- ▶ The padding property can be set using different units such as pixels (px), percentages (%), em units (em), or other relative units. Here's the basic syntax for using the padding property:

```
selector {  
  padding: value;  
}
```

CSS Padding

- You can also specify different padding values for each side using the following syntax:

```
selector {  
  padding-top: value;  
  padding-right: value;  
  padding-bottom: value;  
  padding-left: value;  
}
```

You can also use shorthand notations to set different padding values at once. For example:

```
div {  
  padding: 10px 20px 30px 40px;  
}
```

This shorthand notation sets the padding values in the order top, right, bottom, and left.

CSS Outline

- ▶ In CSS, the outline property is used to create a visible border around an element, typically used to highlight or emphasize the element. Unlike the border property, the outline does not take up space in the layout and is drawn outside the element's dimensions.
- ▶ The outline property can be set with the following syntax:

```
selector {
```

```
  outline: [outline-width] [outline-style] [outline-color];
```

```
}
```

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  border: 2px solid black;
  outline: 5px dashed grey;
  margin: auto;
  padding: 20px;
  text-align: center;
}
</style>
</head>
<body>

<h2>CSS Outline</h2>
<p>This element has a 2px black border and a green outline with a width
of 10px.</p>

</body>
</html>
```

CSS Outline

This element has a 2px black border and a green outline with a width of 10px.

CSS Overflow

In CSS, the overflow property is used to control how content that exceeds the size of an element is handled. It specifies whether to display scrollbars, clip the content, or automatically expand the element to accommodate the overflowing content.

The overflow property can be applied to block-level elements, including `<div>`, `<p>`, and others. Here are some common values for the overflow property:

- ▶ **visible:** This is the default value. Content that exceeds the element's boundaries will be displayed outside the element without any clipping. This can cause the content to overlap with surrounding elements.
- ▶ **hidden:** Content that exceeds the element's boundaries will be clipped and hidden. It won't be visible to the user. This is often used to hide overflowing content.
- ▶ **scroll:** Scrollbars will be added to the element, allowing the user to scroll and view the overflowing content. The scrollbars will always be visible, regardless of whether there is overflowing content or not.
- ▶ **auto:** Scrollbars will be added to the element if the content exceeds its boundaries. If the content fits within the element, no scrollbars will be displayed.

Example

```
<style>
#overflowTest {
  background: #4CAF50;
  color: blue;
  padding: 15px;
  width: 50%;
  height: 100px;
  overflow: visible;
  border: 1px solid #ccc;
  text-align: justify;
}
</style>
```

CSS Overflow

The overflow property controls what happens to content that is too big to fit into an area.

This text is really long and the height of its container is only 100 pixels. Therefore, a scrollbar is added to help the reader to scroll the content. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum. Typi non habent claritatem insitam; est usus legentis in iis qui facit eorum claritatem.

CSS Display

In CSS, the display property is used to control how an element is rendered on a web page. It specifies the type of box used for an HTML element and determines how it interacts with other elements.

Here are some commonly used values for the display property:

- ▶ **block:** This value makes the element generate a block-level box. It takes up the full width of its parent container and starts on a new line. Block-level elements include `<div>`, `<p>`, `<h1>` to `<h6>`, and `<li>`.
- ▶ **inline:** This value makes the element generate an inline-level box. It does not start on a new line and only occupies the space it needs. Inline elements include `<span>`, `<a>`, `<strong>`, and `<em>`.

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
li {
  display: inline;
}
</style>
</head>
<body>

<p>Display a list of links as a horizontal menu:</p>

<ul>
  <li><a href="/html/default.asp" target="_blank">HTML</a></li>
  <li><a href="/css/default.asp" target="_blank">CSS</a></li>
  <li><a href="/js/default.asp" target="_blank">JavaScript</a></li>
</ul>

</body>
</html>
```

Display a list of links as a horizontal menu:

[HTML](#) [CSS](#) [JavaScript](#)

CSS Float

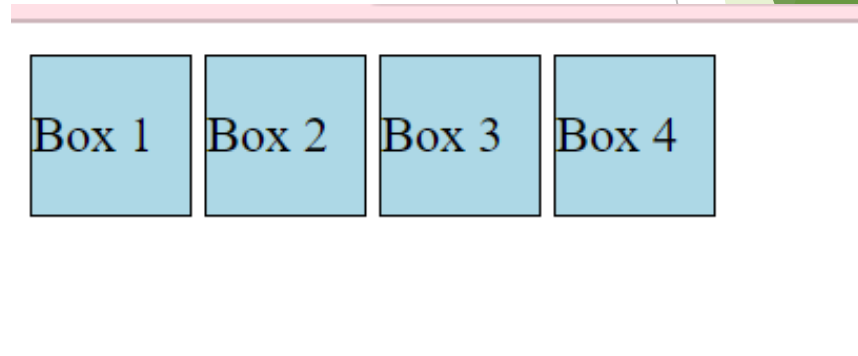
In CSS, the float property is used to position an element along the left or right side of its containing element, allowing other elements to flow around it. When an element is floated, it is taken out of the normal flow of the document.

The float property can be applied to block-level elements. Here are the common values for the float property:

- ▶ **left:** The element is floated to the left side of its containing element. Other elements will flow around it on the right side.
- ▶ **right:** The element is floated to the right side of its containing element. Other elements will flow around it on the left side.
- ▶ **none:** This is the default value. The element is not floated, and it remains in the normal flow of the document. Other elements will not wrap around it.

Example

```
<style>
  div{
    height:50px;
    width:50px;
    border: 1px solid black;
    background-color: lightblue;
    margin: 2px;
    float:left;
  }
</style>
```



CSS Icon

- ▶ The simplest way to add an icon to your HTML page, is with an icon library, such as Font Awesome.
- ▶ Add the name of the specified icon class to any inline HTML element (like `<i>` or `<span>`).
- ▶ To use the Font Awesome icons, go to fontawesome.com, sign in, and get a code to add in the `<head>` section of your HTML page:
- ▶ `<script src="https://kit.fontawesome.com/yourcode.js" crossorigin="anonymous"></script>`

Example

```
<!DOCTYPE html>
<html>
<head>
<script src="https://kit.fontawesome.com/8f61f21cea.js"
crossorigin="anonymous"></script>
</head>
<body>

<i class="fas fa-cloud"></i>
<i class="fas fa-heart"></i>
<i class="fas fa-car"></i>
<i class="fas fa-file"></i>
<i class="fas fa-bars"></i>

</body>
</html>
```



CSS Box Sizing

The box-sizing property in CSS is used to control how the total size of an element is calculated. It affects how the width and height properties are interpreted.

The box-sizing property can have two possible values:

- ▶ **content-box (default):** This is the default behavior of the box model. When box-sizing is set to content-box, the specified width and height values only apply to the content area of the element. The padding, border, and margin are added to the specified width and height, resulting in a larger total size.
- ▶ **border-box:** When box-sizing is set to border-box, the specified width and height values include the content, padding, and border areas of the element. In other words, the dimensions you set will be applied to the entire box, including the content, padding, and border. This ensures that the total size of the element remains consistent, even if you add padding or border to it.

Example

```
<style>
  #container1 {
    box-sizing: border-box;
    padding: 20px;
    border: 30px solid black;
    height: 40%;
    width: 40%;
    margin: auto; /* Center the container */
    background-color: lightblue;
  }
  .container2 {
    box-sizing: content-box;
    padding: 20px;
    border: 30px solid black;
    height: 40%;
    width: 40%;
    margin: auto; /* Center the container */
    background-color: lightblue;
  }
</style>
```

Border Box Example

This is Border box example.

Content box Example

This is content box example.

CSS Position Property

- ▶ The position property specifies the type of positioning method used for an element.
- ▶ There are five different position values:
 - ▶ static
 - ▶ relative
 - ▶ fixed
 - ▶ absolute
 - ▶ sticky
- ▶ Elements are then positioned using the top, bottom, left, and right properties. However, these properties will not work unless the position property is set first. They also work differently depending on the position value.

position: static;

- ▶ In CSS, the `position: static;` property/value is the default positioning behavior for an element. When an element has `position: static;`, it is positioned according to the normal document flow, meaning it will be displayed in its default position without any additional positioning applied to it.
- ▶ Here are some key points about `position: static;`:
- ▶ **Default behavior:** Elements are set to `position: static;` by default if no other positioning property is specified.
- ▶ **Ignored offset properties:** The offset properties such as `top`, `bottom`, `left`, and `right` have no effect on elements with `position: static;`. The element will remain in its default position as determined by the document flow.
- ▶ **Not affected by z-index:** Elements with `position: static;` are not affected by the `z-index` property. They will be layered based on their order in the document flow.

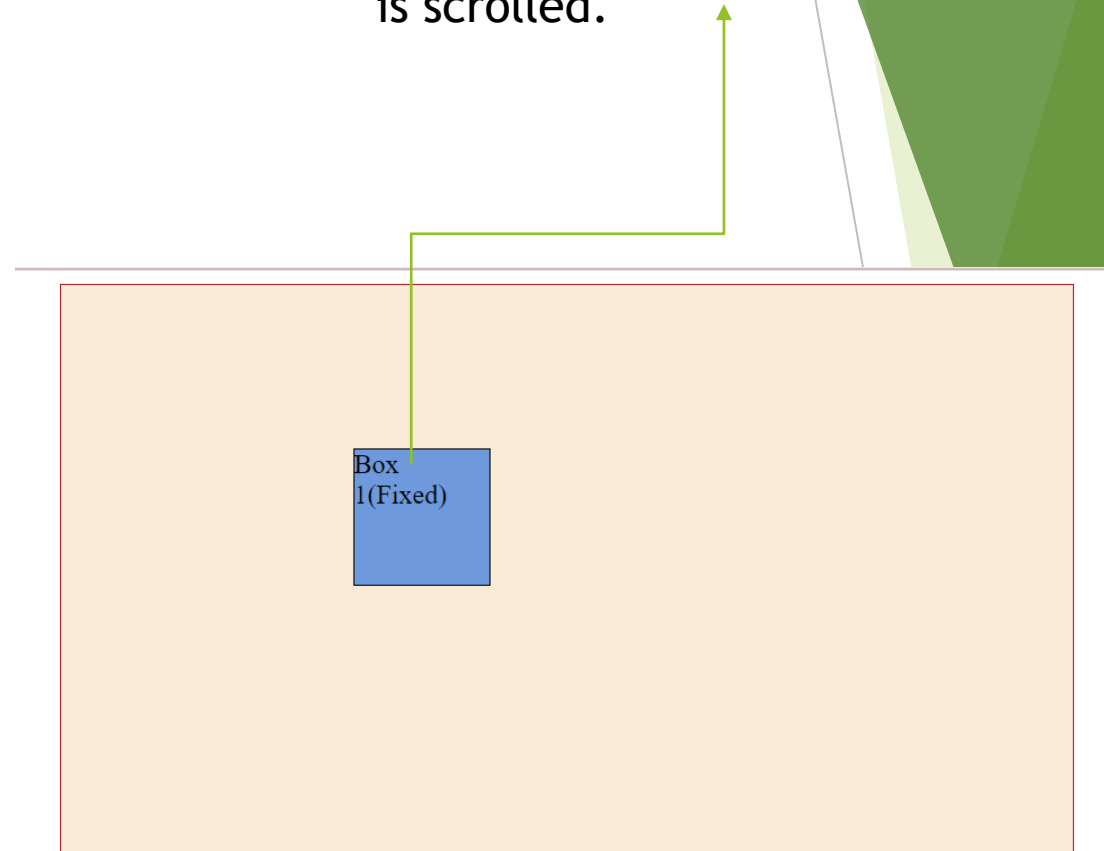
position: fixed;

- ▶ The `position: fixed;` property in CSS is used to position an element relative to the browser window or the viewport, regardless of scrolling. Unlike elements with `position: absolute;`, which are positioned relative to their closest positioned ancestor, elements with `position: fixed;` are positioned relative to the viewport itself.
- ▶ Here are some key points about `position: fixed;`:
- ▶ **Fixed positioning:** An element with `position: fixed;` is removed from the normal document flow and is positioned relative to the viewport. It will stay in the same position even if the page is scrolled.
- ▶ **Ignored by document flow:** Elements with `position: fixed;` do not affect the positioning of other elements. They are essentially "taken out" of the document flow, and other elements will act as if the fixed element doesn't exist.
- ▶ **Relative to the viewport:** The position of a fixed element is determined by its offset properties (`top`, `bottom`, `left`, `right`) in relation to the viewport. For example, `top: 0; left: 0;` would position the element at the top-left corner of the viewport.
- ▶ **Stays visible during scrolling:** Fixed elements remain visible on the screen even when the user scrolls the page. This behavior is commonly used for creating sticky headers, sidebars, or persistent navigation menus.
- ▶ **Overlapping content:** Fixed elements can overlap other elements on the page. If multiple fixed elements have conflicting positions or `z-index` values, the element with the highest `z-index` will be displayed on top.

Example

```
<style>
  .parent{
    height:800px;
    width:600px;
    border: 1px solid ■ brown;
    margin:auto;
    margin-bottom: 900px;
    background-color: ■ antiquewhite;
  }
  .class{
    height:80px;
    width: 80px;
    border:1px solid ■ black;
    background-color: ■ rgb(110, 154, 221);
    margin:5px;
    display:inline-block;
  }
  #child1{
    position:fixed;
    top:100px;
    left:500px;
  }
</style>
```

This content is Fixed. It will stay in the same position even if the page is scrolled.



position: sticky;

- ▶ The `position: sticky;` property in CSS is used to create an element that behaves as `position: relative;` within its parent container until a specified threshold is reached.
- ▶ Once the threshold is reached by scrolling, the element becomes `position: fixed;` and remains fixed in its position within the viewport until the scrolling reaches another specified point.

Example

```
<style>
div.sticky {

    position: sticky;
    top: 0;
    padding: 5px;
    background-color: #6a94e8;
    border: 2px solid #5861dd;
}
</style>
```

Try to **scroll** inside this frame to understand how sticky positioning works.

I am sticky!

In this example, the sticky element sticks to the top of the page (top: 0), when you reach its scroll position.

Scroll back up to remove the stickyness.

Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, malisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, malisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

Position: relative;

- ▶ An element with `position: relative;` is positioned relative to its normal position.
- ▶ Setting the `top`, `right`, `bottom`, and `left` properties of a relatively-positioned element will cause it to be adjusted away from its normal position. Other content will not be adjusted to fit into any gap left by the element.
- ▶ Z-index stacking: Elements with `position: relative;` can be layered using the `z-index` property. The `z-index` value determines the stacking order, with higher values appearing on top of elements with lower values.
- ▶ Child positioning: When a child element of a parent with `position: relative;` is set to `position: absolute;`, it is positioned relative to the parent element rather than the viewport. This is useful for creating relative positioning hierarchies.

Example

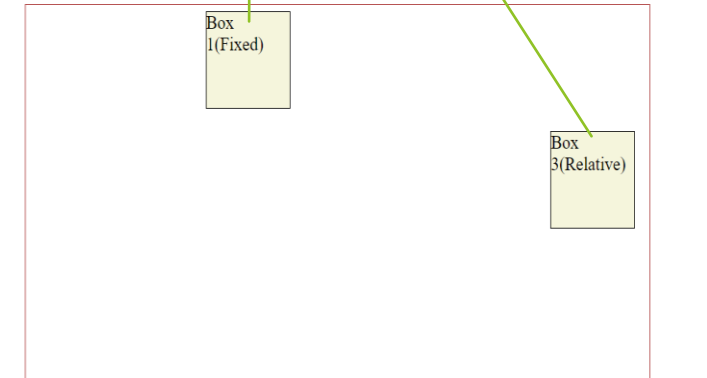
```
#child1{  
  position:fixed;  
  top:100px;  
  left:500px;  
}  
#child2{  
  position:sticky;  
  top:0px;  
}  
#child3{  
  position:relative;  
  top:100px;  
  left:500px;  
}
```

Box 1 and Box 3 are the child of red color box. Box 1 and Box 3 have same spacing top:100px and left:500px. Box 1 sets its position relative to viewport where as Box 3 sets its position relative to its parent.

Box
2(Sticky)

Box
1(Fixed)

Box
3(Relative)



Position: absolute;

- ▶ An element with `position: absolute;` is positioned relative to the nearest positioned ancestor (instead of positioned relative to the viewport, like `fixed`). When a child element of a parent with `position: relative;` is set to `position: absolute;`, it is positioned relative to the parent element rather than the viewport. This is useful for creating relative positioning hierarchies.
- ▶ However; if an absolute positioned element has no positioned ancestors, it uses the document body, and moves along with page scrolling.
- ▶ Note: Absolute positioned elements are removed from the normal flow, and can overlap elements.

Example

```
<style>
div.relative {
  position: relative;
  width: 400px;
  height: 200px;
  border: 3px solid #73AD21;
}

div.absolute {
  position: absolute;
  top: 80px;
  right: 0;
  width: 200px;
  height: 100px;
  border: 3px solid #73AD21;
}
</style>
</head>
<body>

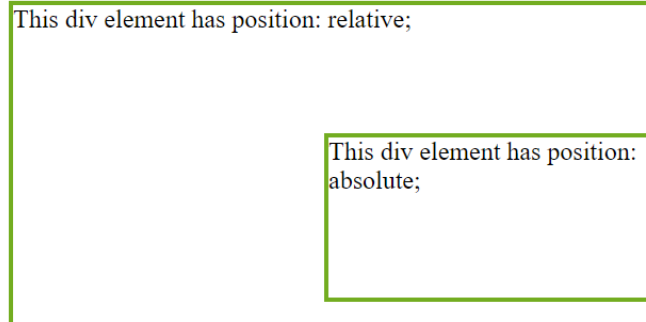
<h2>position: absolute;</h2>

<p>An element with position: absolute; is positioned relative to the nearest positioned ancestor (instead

<div class="relative">This div element has position: relative;
  <div class="absolute">This div element has position: absolute;</div>
</div>
```

position: absolute;

An element with position: absolute; is positioned relative to the nearest positioned ancestor (instead of positioned relative to the viewport, like fixed):



Z-index property

- ▶ When elements are positioned, they can overlap other elements.
- ▶ The z-index property specifies the stack order of an element (which element should be placed in front of, or behind, the others).
- ▶ An element can have a positive or negative stack order:
 - ▶ Eg:
 - ▶ `z-index:-1;`

Box Shadow effect

- ▶ The box-shadow property is used to add shadow effects around an element's frame. You can specify the horizontal and vertical offsets, blur radius, spread radius, and color of the shadow.
- ▶ Syntax:
 - ▶ Box-shadow: x-offset y-offset shadow-color;
 - ▶ Eg. Box-shadow:10px 10px red;
 - ▶ Box-shadow:-10px -10px red;
- ▶ Blur effect: blur parameter defines the blur radius. The higher the number the more blurred the shadow will be
- ▶ Syntax:
 - ▶ Box-shadow: x-offset y-offset blur-amount shadow-color;
 - ▶ Eg. Box-shadow:10px 10px 20px red;

Main menu

Main menu

Main menu

Box Shadow effect

- ▶ **Spread radius:** Spread parameter defines the spread radius. A positive value increases the size of size of the shadow, a negative value decreases the size of the shadow

- ▶ **Syntax:**

- ▶ Box-shadow: x-offset y-offset blur-amount spread-radius shadow-color;
- ▶ Eg. Box-shadow:10px 10px 20px 30px blue;

Inset parameter: Inset parameter changes the shadow from an outer shadow to an inner shadow

- ▶ **Syntax:**

- ▶ Eg. Box-shadow:10px 10px 30px blue inset;

Main menu

Main
menu

Box Shadow effect

Multiple Shadow effect

- ▶ Syntax:

- ▶ Eg. Box-shadow:10px 10px 10px 20px red, 10px 10px 10px 20px blue;



Main menu

Text Shadow effect

- ▶ The text shadow property is used to add shadow effects around an text. You can specify the horizontal and vertical offsets, blur radius, spread radius, and color of the shadow same as box shadow effect.
- ▶ Syntax:
 - ▶ `text-shadow: x-offset y-offset shadow-color;`
 - ▶ Eg. `text-shadow:10px 10px blue;`

This is text shadow effect

Text Effects

- ▶ **text-overflow:** property to handle overflow of text content inside the specific element
 - ▶ snip value: it will crop the content that are only fit inside the element and hide the extra content
 - ▶ ellipsis value: it will add few dots at end of element if there are extra contents hidden due to overflow
 - ▶ `P{text-overflow:ellipsis;}`
- ▶ **word-wrap:** word wrap property allows long words to be able be broken and wrap into the next line. If a word is too long to fit within an area, it expands outside.
 - ▶ The word wrap property allows us to force the text to wrap
 - ▶ Eg: `p{word-wrap:break-word;}`

This is very long paragraph Thi

This is very long paragraph ...

This is very long
paragraphparagraphparagraph
This is very long paragraph
This is very long paragraph
This is very long paragraph
This is very long paragraph
This is very long paragraph

This is very long
paragraphparagraphparagraph
paragraphparagraphparagraphpa
ragraphparagraphparagraphpar
agraph This is very long
paragraph This is very long
paragraph This is very long
paragraph This is very long
paragraph This is very long
paragraph

Text Effects

- ▶ **word-break:** word break property specifies line breaking rules
 - ▶ P{word-break:break-all;}

[illegible]

- ▶ **writing-mode:** writing mode property specifies whether lines of text are laid out horizontally or vertically. It can have values: horizontal-tb, vertical-rl.
 - ▶ Eg. P{writing-mode:vertical-rl}

this is vertical

Basics of Responsive Web Designs

- ▶ Web pages can be viewed using many different devices: desktops, tablets, and phones. Web page should look good and easy to use regardless of the device. Responsive web design makes our web page look good on all devices. Web pages should not leave out information to fit smaller devices, but rather adopt its content to fit any device. It is called responsive web design when we use css and html to resize, hide, shrink, enlarge or move the content to make it look good on any screen.

Media Queries

- ▶ Media queries in CSS3 extended the CSS2 media types idea: Instead of looking for a type of device, they look at the capability of the device.
- ▶ Media queries can be used to check many things, such as:
 - ▶ width and height of the viewport
 - ▶ width and height of the device
 - ▶ orientation (is the tablet/phone in landscape or portrait mode?)
 - ▶ resolution
 - ▶ Using media queries are a popular technique for delivering a tailored style sheet to desktops, laptops, tablets, and mobile phones (such as iPhone and Android phones).

Media Query Syntax

- ▶ A media query consists of a media type and can contain one or more expressions, which resolve to either true or false.

```
@media not|only mediatype and (expressions) {  
    CSS-Code;  
}
```

eg. @media only screen and (max-width:600px)

```
{  
    div{  
        width:360px;  
    }  
}
```

eg. media only screen and (orientation:landscape)

- ▶ The result of the query is true if the specified media type matches the type of device the document is being displayed on and all expressions in the media query are true. When a media query is true, the corresponding style sheet or style rules are applied, following the normal cascading rules.

Responsive framework

- ▶ A responsive framework, also known as a responsive web design framework, is a collection of pre-built CSS and JavaScript components that help developers create responsive and mobile-friendly websites more easily. These frameworks provide a set of reusable UI elements, grid systems, typography styles, and other components that are designed to adapt and adjust based on different screen sizes and devices.
- ▶ Here are some popular responsive frameworks:
- ▶ **Bootstrap:** Bootstrap is one of the most widely used responsive frameworks. It provides a comprehensive set of CSS and JavaScript components, including a responsive grid system, navigation bars, buttons, forms, and more. Bootstrap is known for its simplicity, flexibility, and extensive documentation.
- ▶ **Foundation:** Foundation is another popular responsive framework. It offers a robust grid system, a wide range of UI components, responsive navigation menus, and various layout options. Foundation emphasizes customization and provides a flexible framework that can be tailored to specific project needs.

Responsive framework

- ▶ **Bulma:** Bulma is a lightweight and modern responsive framework. It utilizes a flexible grid system and provides a rich set of CSS classes to create responsive layouts. Bulma also includes components such as forms, buttons, cards, and navigation menus. It is known for its simplicity and ease of customization.
- ▶ **Material-UI:** Material-UI is a responsive framework that implements the Material Design guidelines developed by Google. It offers a rich collection of React components for building responsive web applications. Material-UI provides ready-to-use components such as buttons, cards, form elements, and navigation components, all following the Material Design principles.
- ▶ **Tailwind CSS:** Tailwind CSS is a utility-first CSS framework that enables rapid development and customization. It provides a large number of CSS utility classes that can be composed to create responsive designs. Tailwind CSS offers flexibility and allows developers to build unique layouts without being tied to predefined styles or components.

These frameworks provide a foundation for building responsive websites by offering a set of pre-designed and tested components. They help streamline development, improve consistency, and ensure a seamless user experience across different devices. However, it's important to consider the specific requirements and goals of your project before choosing a responsive framework.

Introduction to Bootstrap

- ▶ Bootstrap is a free and open source most popular CSS framework for developing responsive and mobile first websites. Bootstrap 5 is the newer version boot strap. Bootstrap makes things a whole lot easier. Bootstrap can help speed up development times, while maintaining quality and consistency across the site. There are so many boot strap components likes header, navigation, cards slider etc while we can use easily and save our time. With the help of these we don't need to spend hours to design any element.
- ▶ You can wither download and install the bootstrap library or copy the specific bootstrap code from live server.